

RESUMO DO MÓDULO

Grupo de Rotas

No Express.js, é possível agrupar rotas relacionadas em um "grupo de rotas". Isso é feito utilizando a classe `Router()` do próprio Express. Essa abordagem permite que você organize o código em módulos separados, facilitando a manutenção e a escalabilidade do seu projeto.

Ao usar grupos de rotas, você separa as funcionalidades de diferentes partes do aplicativo em arquivos independentes, como `userRoutes.js` e `postRoutes.js`. Cada arquivo contém um conjunto de rotas relacionadas, e, no final, essas rotas são conectadas ao arquivo principal (`app.js`) usando o método `use()`.

Por que Usar Grupos de Rotas?

Se todas as rotas de um aplicativo fossem definidas no mesmo arquivo (`app.js`), o código ficaria confuso e difícil de gerenciar, especialmente em projetos maiores. Separar as rotas em grupos torna o código mais organizado e modular. Imagine um projeto com dezenas de páginas e rotas diferentes; ter todas elas no mesmo lugar seria inviável.

Criando Grupos de Rotas no Express

Vamos criar um exemplo prático para ilustrar como organizar rotas para usuários e postagens em arquivos separados. Vamos supor que o projeto tenha um `app.js` e dois novos arquivos para cada grupo de rotas: `userRoutes.js` e `postRoutes.js`.

1. Criando o Arquivo `userRoutes.js`

Esse arquivo contém as rotas relacionadas a usuários:

```
// userRoutes.js

const express = require('express');

const userRouter = express.Router();


userRouter.get('/', (req, res) => {

  res.send('Caiu na rota de usuários');

});


module.exports = userRouter;
```

Aqui, a classe Router() é usada para criar uma instância de roteador chamada userRouter. Em seguida, definimos a rota para a raiz (/) desse grupo, que responde com a mensagem "Caiu na rota de usuários".

2. Criando o Arquivo postRoutes.js

Esse arquivo contém as rotas relacionadas a postagens:

```
// postRoutes.js

const express = require('express');

const postRouter = express.Router();


postRouter.get('/', (req, res) => {

  res.send('Caiu na rota de posts');

});


module.exports = postRouter;
```

Aqui, criamos a instância postRouter e definimos a rota raiz (/) do grupo de rotas para postagens, que responde com a mensagem "Caiu na rota de posts".

3. Associando os Grupos de Rotas no app.js

O arquivo app.js será responsável por importar e conectar os grupos de rotas criados:

```
// app.js

const express = require('express');

const app = express();

const userRouter = require('./userRoutes.js'); // Importando o grupo de rotas de usuários
const postRouter = require('./postRoutes.js'); // Importando o grupo de rotas de postagens


// Associar os grupos de rotas de usuários
app.use('/users', userRouter);


// Associar os grupos de rotas de postagens
app.use('/posts', postRouter);


// Inicializando o servidor
app.listen(3000, () => {
  console.log('Servidor em execução em http://localhost:3000/');
});
```

Explicação do Código

1. Importando os Grupos de Rotas:

- userRouter é importado do arquivo userRoutes.js.
- postRouter é importado do arquivo postRoutes.js.

2. Associando os Grupos de Rotas:

- `app.use('/users', userRouter);` indica que todas as rotas definidas em `userRouter` estarão dentro do caminho `/users`.
- `app.use('/posts', postRouter);` faz o mesmo para o grupo de rotas `postRouter`.

3. Comportamento das Rotas:

- Ao acessar `http://localhost:3000/users`, você verá a resposta "Caiu na rota de usuários".
- Ao acessar `http://localhost:3000/posts`, você verá a resposta "Caiu na rota de posts".

Expandindo o Exemplo

Podemos adicionar mais rotas a cada grupo para ilustrar a flexibilidade desse método. Vamos expandir o grupo de rotas de usuários para incluir uma rota de salvamento (`/save`):

Atualizando o `userRoutes.js`

```
const express = require('express');
const userRouter = express.Router();

// Definindo a rota raiz para usuários
userRouter.get('/', (req, res) => {
  res.send('Caiu na rota de usuários');
});

// Definindo uma rota adicional para salvar usuários
userRouter.get('/save', (req, res) => {
```

```
res.send('Salvando usuário');  
});
```

```
module.exports = userRouter;
```

Agora, ao acessar <http://localhost:3000/users/save>, você verá a mensagem "Salvando usuário".

Vantagens de Usar Grupos de Rotas

1. Organização: Facilita a leitura e manutenção do código, especialmente em projetos maiores.
2. Modularidade: Permite adicionar ou remover grupos de rotas facilmente sem interferir no restante do código.
3. Escalabilidade: Novos módulos e funcionalidades podem ser adicionados sem criar um arquivo central gigantesco.

Conclusão

Usar grupos de rotas com `express.Router()` é uma prática recomendada para organizar o código de maneira mais limpa e modular. Essa abordagem ajuda a manter o projeto Express.js organizado e facilita a manutenção conforme o aplicativo cresce. Ao separar as rotas em módulos independentes, você consegue gerenciar diferentes funcionalidades de forma mais eficiente, mantendo a legibilidade e a modularidade do código.

Essa técnica é especialmente útil em projetos grandes, onde a manutenção de rotas em um único arquivo se torna impraticável. Com grupos de rotas, é possível gerenciar cada parte do aplicativo de maneira isolada, garantindo um fluxo de trabalho mais organizado e estruturado.
